

WAKE

Location-Based Wake Alarm

WAKE is an Android app that wakes you up automatically when you physically arrive somewhere — useful for commutes, bus/train naps, or any time you don't want to miss a stop. You pick a destination on a map, pick a sound, and the app tracks your live location in the background and rings when you enter a set radius, even with the phone locked.

What It Does

- 1. Pick a destination** — search or tap a location on an interactive map, then set a "wake radius" (how close you need to be before it triggers).
 - 2. Pick a sound** — choose any audio file from your phone to use as the alarm.
 - 3. Arm and go** — the app tracks your live GPS position, shows live distance on a radar-style display, and fires a full-screen alarm with sound and vibration once you're within the radius — even in the background with the screen off.
-

Tech Stack

Layer	Technology
UI / logic	HTML, CSS, JavaScript (vanilla, no framework)
Maps & location search	Google Maps JavaScript API, Places API, Geocoding API
Native Android wrapper	Capacitor (capacitorjs.com)
Background GPS tracking	@capacitor-community/background-geolocation
Native alarm notifications	@capacitor/local-notifications
Build tooling	Node.js, npm, Android Studio, Gradle

The app is built as a **web app first** (HTML/CSS/JS, installable as a standalone PWA), then wrapped into a real native Android app so it can keep tracking location and ring an alarm even when the phone is locked — something a browser alone can't do.

Key Terms Explained

- **PWA (Progressive Web App)** — a website that can be installed on a phone like a regular app, using a manifest.json and a service worker for offline support. Works well while open, but browsers restrict what a PWA can do in the background.
- **API (Application Programming Interface)** — a defined way for one piece of software to request data or functionality from another. Here, the app calls Google's Maps, Places, and Geocoding APIs to show maps, search locations, and convert coordinates into readable addresses.

- **API key** — a unique code that identifies and authorizes an app to use a given API, restricted here to specific APIs and specific app origins for security.
 - **Capacitor** — a toolkit that takes a web app (HTML/CSS/JS) and wraps it inside a real native app shell, giving it access to native device features (GPS, notifications, files) that a browser can't normally provide.
 - **Node.js / npm / npx** — Node.js is a JavaScript runtime used to run build tools on a computer (not in a browser). npm installs project dependencies; npx runs installed command-line tools.
 - **Gradle** — the build system Android uses to compile a project (including Capacitor-wrapped apps) into an installable .apk file.
 - **Android Studio** — Google's official IDE for building, compiling, and running Android apps, including the Capacitor-generated project here.
 - **Geolocation API** — the browser/OS-level interface used to read GPS coordinates.
 - **Haversine formula** — the mathematical formula used to calculate the real-world distance between two GPS coordinates (used here to measure distance to the destination).
 - **Adaptive icon** — Android's modern app icon format, built from separate foreground and background image layers so the icon displays correctly across different phone launcher shapes (circle, square, rounded square).
-

Project Structure

```
wake/
├── www/                                # the core web app
│   ├── index.html
│   ├── css/style.css
│   ├── js/app.js                      # state, geolocation, distance calc, alarm logic
│   ├── js/map.js                    # Google Maps + Places integration
│   ├── manifest.json
│   └── service-worker.js
├── android/                          # native Android project (generated by Capacitor)
├── capacitor.config.json
└── package.json
```

Build Steps

1. Enable **Maps JavaScript API**, **Places API**, and **Geocoding API** on a Google Cloud project, and generate a restricted API key.
2. Add the API key to `www/js/map.js`.
3. Install **Node.js** and **Android Studio**.
4. Run `npm install` to pull in Capacitor and its plugins.
5. Run `npx cap add android` to generate the native Android project.
6. Run `npx cap sync android` to copy the web app into the native project.

7. Open the project in Android Studio (`npx cap open android`), connect an Android phone via USB with Developer Options + USB Debugging enabled, and run the app directly onto the device.
 8. Customize the app name (`strings.xml`) and icon (Android Studio's Image Asset tool) as desired.
-

Known Limitation

Android's Do Not Disturb settings can, in some cases, still suppress the alarm notification's sound. For guaranteed ringing through silent/DND mode (like a native alarm clock app), a further native module using Android's `AlarmManager` and a full-screen intent would be needed — not included in this version.

License

Personal project — add a license of your choice if publishing publicly.